

SPHEREX

College Server Setup & Deployment Instructions

Document purpose

This guide describes a practical deployment process for installing a college-specific SPHEREX environment. It is based on the current project architecture: Next.js/React web application, Prisma + SQLite persistence, Firebase synchronization, and a Node.js WebRTC signaling service. The Android application is packaged using Capacitor.

1. Recommended Deployment Architecture

College Server

Next.js application + API + Prisma/database + configuration

Optional Voice Server

Node.js WebRTC signaling service

Cloud Services

Firebase Authentication / Firestore / Realtime Database when enabled

Users

Desktop browsers + Android application

2. Server Prerequisites

- A supported Node.js runtime and npm
- Git or another method to transfer the application source/build
- Sufficient server storage for application, database and logs
- Network access for required Firebase/cloud services if synchronization is enabled
- HTTPS/TLS for production use, especially for authentication and browser microphone/WebRTC features
- A process manager/service mechanism for long-running Node.js services
- A backup location for the college database and configuration

3. Application Installation

Copy the SPHEREX project to the college server and open a terminal in the project directory.

Install dependencies

```
npm install
```

Prepare Prisma

```
npx prisma db push  
npx prisma generate
```

Seed only when required

```
npm run prisma:seed
```

Use seeding for a new environment only after confirming that it will not overwrite or duplicate institutional data.

4. Configure Institution Data

- Replace V.S.B.-specific branding, campuses, faculty and sample data with the customer's institution configuration.
- Create the college's campuses and departments.
- Create institution-specific admin, counselor and faculty accounts.
- Configure admission stages, lead sources and quotas.
- Remove development/sample credentials before production go-live.

5. Firebase / Cloud Configuration

The current project contains Firebase initialization and synchronization components. If the college uses Firebase, create or assign a separate Firebase project/environment for that institution and configure the application's Firebase settings securely. Do not reuse another college's Firebase credentials or student data.

6. Start the Web Application

```
npm run dev
```

This command is intended for development/testing. For production, use the project's production build/start process and a suitable service manager or hosting platform.

7. WebRTC Voice Service

The current architecture contains a separate Node.js signaling server under `webrtc-voice-system/` and documents port 5000 for signaling. Start it using the project's configured script:

```
npm run webrtc
```

For production, place the service behind appropriate HTTPS/WSS networking and restrict access to authorized users.

8. Android App Deployment

The current project uses Capacitor Android packaging. The documented build flow is:

```
npm run build:mobile  
npm run cap:open  
npm run build:apk
```

- Update the application identifier/package name for the college if required.
- Replace app name, icon and branding with the college-specific configuration.
- Set the production web/API endpoint used by the app.
- Build and test the APK on supported Android devices.
- Distribute the signed release APK through the college's approved internal method or app-management system.

9. Production Security Checklist

- Use HTTPS for the production web application.
- Use strong, unique administrative credentials.
- Never keep default/demo passwords in production.

- Store Firebase/API secrets in protected environment configuration, not source code.
- Restrict server ports and administrative interfaces with firewall rules.
- Back up the database regularly and test restoration.
- Separate each college's database/Firebase environment or enforce strict tenant isolation.
- Enable audit logging for administrative actions where supported.
- Review microphone/call recording permissions and institutional policies before enabling voice features.

10. Testing Before Go-Live

- Admin login and role permissions
- Counselor and faculty access boundaries
- Create Quick Lead
- Create New Application
- CSV import
- Search/filter student records
- Admission pipeline transitions
- Faculty lead allocation
- Marketing source attribution
- Payment/application status
- Firebase synchronization
- Offline/local persistence behavior where applicable
- Voice calling/recording if enabled
- Android login and API connectivity
- Backup and restore

11. Go-Live Procedure

- Freeze configuration and take a backup.
- Create production admin accounts securely.
- Verify college branding and domain.
- Run final role/access tests.
- Import approved existing data if required.
- Confirm web and Android endpoints.
- Train administrators, counselors and faculty.
- Obtain college sign-off.
- Enable production access and monitor the first operational period.

12. Support & Maintenance

Maintain a separate configuration record for each college: institution name, campuses, enabled modules, deployment type, application version, database backup schedule, Firebase environment, Android version and

support contact. Keep customer data isolated and document every production change.

Architecture note: The knowledge base documents SQLite as the current Prisma database and Firebase as a synchronization layer. Before selling college-server deployments at scale, validate production database concurrency, backup/restore, authentication security, tenant isolation and WebRTC networking for the target infrastructure.